

BioEval Reproducibility Report

Computational Simulation Papers — Generated June 6, 2026

This report presents structured reproducibility evaluations for 14 computational simulation projects. Each is scored across six dimensions: Data Disclosure, Dataset Resolvability, Code Availability, Traceability, Simulation Clarity, and Reproducibility Package Quality. Papers are ordered by overall score.

All Papers — Score Summary							
Paper	Score	Disclosure	Dataset	Code	Trace	Sim	Repro
Ant Simulation — Pygame Stigmergy	85.4	92	98	83	88	75	72
Firefly Synchronization Simulation (JS...	73.6	82	78	78	75	68	52
BeeStack: An Evidence-Typed Scaffold f...	73.1	72	58	88	75	78	62
The Ant Stack: Reproducible scientific...	72.2	72	68	92	62	66	82
Termite Colony — Unity 3D Multiagent	71.6	82	78	62	75	72	48
Bee Swarm Simulation	68.1	72	68	75	62	58	82
Rust Ant Colony Simulation	64.3	82	75	72	48	52	58
ACO Robot Path Planning (Python)	54.5	52	58	68	62	50	28
Ant Colony Optimization — React/Canvas	52.7	58	62	72	42	38	50
Locas Ants — Lua/Love2D Ant Colony	51.6	48	55	82	38	38	62
Ants Sandbox (TypeScript/web)	51.1	48	58	78	38	40	55
Symbiants — Ant Colony Sim (Rust)	43.8	38	42	78	30	32	58
swarm-abc — Artificial Bee Colony Simu...	41.9	38	48	62	35	40	28
Hardware-Accelerated Ant Colony Swarm ...	37.8	32	38	58	35	30	40

Ant Simulation — Pygame Stigmergy

Source: <https://github.com/lrdcxdes/ant-simulation>

Evaluated: June 3, 2026

85.4

Overall /100



Summary

This is a publicly available agent-based simulation project with clear installation instructions and an MIT license, but it falls well short of scientific reproducibility standards. The README names five tunable parameters (ANT_COUNT, EVAPORATION_RATE, NEST_GRAVITY_STRENGTH, MAX_PATIENCE, PANIC_DURATION) as the core 'laws of physics' yet reports none of their actual values, leaving no way to reproduce the specific behavior shown in the demo video. There are no citations for the biological or algorithmic basis of any parameter choices, no version tag or archived DOI, no tests or CI, and the 'Mental Fatigue' mechanism is described as a novel discovery without any formal validation or comparison to baseline approaches. Information flow through the pheromone field — the core phenomenon of the system — is never quantified in information-theoretic terms despite being directly amenable to entropy and transfer-entropy analysis.

Positive Findings

- Source code is publicly accessible on GitHub under the MIT License
- A dependency manifest (requirements.txt) listing Pygame, NumPy, and SciPy is included
- A demo video (demo.mp4) is committed to the repository, illustrating typical simulation behavior
- Five key tunable parameters are named explicitly in the README (ANT_COUNT, EVAPORATION_RATE, NEST_GRAVITY_STRENGTH, MAX_PATIENCE, PANIC_DURATION) and described as user-adjustable
- Step-by-step clone and run instructions are provided, covering environment setup in three commands
- The simulation is self-contained in a single file (sim.py) with no external data dependencies, which simplifies re-execution
- The generative rules (stigmergy, evaporation/diffusion, nest gravity, mental fatigue/panic) are described conceptually in the README

Gaps Identified

- No actual numeric values are reported for any of the five named parameters, making exact reproduction of the demo behavior impossible
- No version tag, pinned commit hash, or archived DOI (e.g. Zenodo) is provided; the repository has 0 published releases and 0 tags
- The cd target in the clone instructions ('cd emergent-mind') does not match the repository name ('ant-simulation'), which will cause a failure for users who copy-paste the instructions verbatim
- No citations are provided for the biological or algorithmic basis of any simulation rule or parameter, including the stigmergy model, evaporation dynamics, or nest-gravity heuristic
- The 'Mental Fatigue and Panic' mechanism is claimed as a novel scientific discovery but is presented with no formal validation, no comparison to baseline (e.g., standard ACO without fatigue), and no quantitative performance metrics
- No random seed is documented or exposed as a parameter, making stochastic runs non-reproducible even with correct parameter values
- No tests, CI/CD workflows, or automated checks are present in the repository
- No supplementary figures, tabulated results, or analytical outputs are provided beyond a single demo video
- Information flow through the pheromone field is never quantified (e.g., Shannon entropy of pheromone distributions, mutual information between ant states, channel capacity of the stigmergic signal) despite being the central phenomenon
- [Critic] No tests directory or CI/CD workflows are detected, meaning there is no automated verification that sim.py executes correctly or that the listed dependencies in requirements.txt are sufficient and compatible.
- [Critic] The repository has 0 published releases and 0 tags, so there is no versioned snapshot corresponding to any specific paper submission or result — re-running the code at a later commit may not reproduce the exact behavior described.
- [Critic] With only 2 stars and 1 open issue, community validation of the simulation's correctness is minimal, weakening any implicit claim of peer-reviewed or community-vetted software quality.

Recommendations

- Publish a versioned release on GitHub and archive the release on Zenodo to obtain a citable DOI; reference this DOI in any associated manuscript
- Fix the clone instructions so the cd command matches the actual repository name ('cd ant-simulation')
- Add a configuration file (e.g., config.yaml or a params section in the README) that explicitly states every default parameter value used to generate the demo video, including the random seed
- Expose the random seed as a command-line argument or config entry so that users can reproduce specific runs exactly
- Add at least one citation (or a theoretical derivation) for each core mechanism: the pheromone evaporation/diffusion model, the nest-gravity heuristic, and any prior 'patience'/'fatigue' mechanisms in ant-colony optimization literature
- Provide a quantitative evaluation of the Mental Fatigue mechanism: compare foraging efficiency (e.g., food-return rate, convergence time

to shortest path) with and without the fatigue mechanism across multiple random seeds

- Add a requirements.txt with pinned version numbers (e.g., pygame==2.5.2) to ensure environment reproducibility
 - Quantify information flow in the system: compute pheromone-field entropy over time, mutual information between ant positions and pheromone concentrations, or transfer entropy between agent states, and include these as supplementary figures or an analysis script
 - Add a minimal test script (e.g., a headless 100-step run that checks pheromone grid shape and ant count) to enable automated CI validation
-

Firefly Synchronization Simulation (JS)

Source: <https://github.com/lax4mike/firefly>

Evaluated: June 3, 2026

73.6

Overall /100



Summary

This repository hosts a JavaScript/Arduino firefly synchronization simulation with a public GitHub presence and a live demo URL, providing basic accessibility. However, critical reproducibility infrastructure is absent: there is no license, no versioned release or pinned commit tag, no CI/testing framework, no Dockerfile, and the README is minimal (~42 bytes). While the Mirollo-Strogatz 1990 PDF and a firefly algorithms PDF are bundled as references, simulation parameters, initial conditions, and random seeds are not explicitly documented, making exact reproduction of specific simulation behaviors unclear. The repository has not been updated since September 2021 and lacks any formal reproducibility package, changelog, or supplementary output data.

Positive Findings

- Source code for the firefly synchronization simulation is publicly accessible on GitHub at [lax4mike/firefly](https://github.com/lax4mike/firefly)
- A live interactive demo is hosted at mikelambert.me/firefly, providing immediate access to simulation outputs
- The Mirollo-Strogatz 1990 foundational paper is included directly in the repository as a PDF, establishing the theoretical basis
- A supplementary firefly algorithms PDF is included, partially documenting methodological choices
- Build tooling is present via `gulpfile.js` and `package.json`, providing a dependency manifest that enables partial build reproducibility
- The repository includes Arduino (C++) code alongside JavaScript, indicating both hardware and web simulation modalities are supported
- The repository is public and forkable with 85 commits of history, allowing inspection of development trajectory

Gaps Identified

- No software license is detected, making reuse, modification, and redistribution legally ambiguous
- No versioned releases or Git tags exist, so there is no pinned reference point for reproducing a specific simulation state
- The README is approximately 42 bytes — essentially a title and URL — providing no documentation of how to install, run, or configure the simulation
- No CI pipeline (GitHub Actions or equivalent) and no tests directory are present, so correctness of builds is unverified
- Simulation parameters (coupling strength, oscillator frequency distributions, initial phase conditions, random seeds) are not documented in any config file or README
- No supplementary output data (e.g., synchronization time-series CSVs, result tables) are present to allow validation of simulation outputs
- No Zenodo/DOI archive snapshot exists, meaning the codebase could disappear or change without a stable citeable reference
- No Dockerfile or containerized environment specification is provided, requiring manual environment reconstruction
- Repository has been inactive since September 2021 with one open issue unresolved
- No changelog or release notes document what changed across the 85 commits
- [Critic] No license is detected on the repository, so the claim that it is available under a 'permissive visibility setting' does not imply permissive reuse rights; the code is publicly viewable but its reuse terms are legally ambiguous without a license.
- [Critic] The README is only ~42 bytes, providing essentially no documentation of how to build, run, or reproduce the simulation, which undermines any strong reproducibility claim.
- [Critic] The repository has not been updated in ~58 months (last push 2021-09-12), and the live demo URL may no longer be functional; longevity of the demo as evidence of availability is not guaranteed.
- [Critic] No tests directory and no CI are detected, meaning correctness of the simulation implementation relative to the cited Mirollo-Strogatz 1990 parameters cannot be automatically verified.

Recommendations

- Add a permissive open-source license (e.g., MIT or Apache 2.0) to clarify reuse rights
- Create a tagged Git release (e.g., v1.0.0) and deposit the repository to Zenodo to obtain a citable DOI and stable snapshot
- Expand the README substantially: include installation instructions, parameter descriptions, example run commands, and a description of the simulation model and its relationship to Mirollo-Strogatz 1990
- Document all key simulation parameters (oscillator count, coupling epsilon, frequency range, initial phase distribution, random seed) in a configuration file (e.g., `config.json`) with inline comments
- Add at least one automated test (e.g., verifying synchronization is achieved within N steps for a fixed seed) and configure GitHub Actions CI to run it on each push
- Pin all npm dependency versions in `package-lock.json` and commit it, ensuring reproducible dependency installation
- Add a Dockerfile or devcontainer specification to allow one-command environment setup

- Include representative simulation output data (e.g., phase time-series for a canonical parameter set) as CSV or JSON files so results can be compared against a known reference
 - Add information-theoretic analysis quantifying mutual information or transfer entropy between oscillators as synchronization emerges, which would substantially increase the scientific contribution
-

BeeStack: An Evidence-Typed Scaffold for Whole-Colony Honeybee Simulation

Source: <https://zenodo.org/records/20420557>

Evaluated: June 3, 2026

73.1

Overall /100



Summary

BeeStack demonstrates commendable transparency infrastructure: source is publicly archived on Zenodo (DOI: 10.5281/zenodo.20420557) with a tagged GitHub release (v1.0.0), a pyproject.toml dependency manifest, and a tests directory, and the repository source files contain multiple resolvable DOIs linking empirical datasets (BeeBrain, figshare, Dryad, journal articles) to specific pipeline documents. However, reproducibility is meaningfully limited by the absence of a detected license in the GitHub repository (contradicting the MIT claim on Zenodo), no CI/CD pipeline, a minimal README (~259 bytes), no pinned dependency versions, and validation that is entirely internal (config-band self-test) with no external benchmark dataset cited. The waggle dance and other colony-level communication phenomena — where information-theoretic quantification would be scientifically central — are described mechanistically without any Shannon entropy, mutual information, or channel-capacity analysis.

Positive Findings

- Source code publicly archived on Zenodo with DOI 10.5281/zenodo.20420557 and MD5 checksum, providing a citable, versioned snapshot
- Tagged GitHub release v1.0.0 exists with a linked Zenodo supplement, enabling pinned reproducibility
- Repository source-file scan confirms DOIs embedded in docs/beebrain_data_pipeline.md and manuscript/ files, including figshare (10.6084/m9.figshare.24715977.v1), multiple Dryad accessions (10.5061/dryad.*), and journal article DOIs — demonstrating genuine per-dataset traceability within the codebase
- 9 of 10 scanned DOIs resolve publicly (only 10.1186/s12864-019-5639-3 is unresolved); Dryad and figshare datasets are confirmed public
- pyproject.toml dependency manifest present, enabling partial environment reconstruction
- Tests directory detected, indicating some automated verification of internal logic
- Gaps in fidelity are explicitly catalogued rather than hidden, a notable transparency practice
- MIT License declared on Zenodo record, providing clear reuse terms
- Combined manuscript PDF deposited with MD5 checksum alongside source archive
- Every quoted number is described as traceable from config to artifact to manuscript, with simulation parameters embedded in config files in the repository

Gaps Identified

- GitHub API detects NO license file in the repository itself, contradicting the MIT claim on Zenodo — users cloning from GitHub encounter an ambiguous licensing situation
- README is only ~259 bytes, providing insufficient onboarding documentation for reproducing any simulation layer
- No CI/CD (GitHub Actions or equivalent) detected, so there is no automated verification that the codebase remains functional
- Dependency versions in pyproject.toml are not pinned (not confirmed from API), creating environment drift risk; no lockfile detected
- Validation is entirely a config-band self-test with no external benchmark dataset cited, making biological realism assessment impossible from outside the project
- One DOI (10.1186/s12864-019-5639-3) is unresolved in both Crossref and DataCite
- No Dockerfile or containerized environment detected, requiring manual environment reconstruction
- FlyBody and MuJoCo dependency versions are not specified in the Zenodo record or README
- No waggle dance or colony-communication channel capacity, Shannon entropy, or mutual information quantification despite these being the scientifically central phenomena
- Per-layer dataset accessions and parameter source citations are not surfaced in the Zenodo landing page, requiring users to navigate deep into repository source files
- [Critic] The repository has 0 stars and 2 open issues with only a ~259-byte README, indicating minimal external scrutiny and very sparse documentation — the manuscript's implicit claim of research-grade, well-documented software may be overstated relative to actual repository maturity.
- [Critic] No CI/GitHub Actions workflows are detected, meaning the automated reproducibility pipeline (if any) is not publicly verifiable; any claim that results are automatically reproducible from the repository cannot be confirmed from available evidence.
- [Critic] The unresolved DOI 10.1186/s12864-019-5639-3 was not found in Crossref or DataCite; while this alone may not be a defect, it represents a citation embedded in repository source files (docs/beebrain_data_pipeline.md) whose target cannot be confirmed, weakening provenance traceability for that specific reference.

Recommendations

- Add a LICENSE file directly to the GitHub repository root to resolve the discrepancy with the Zenodo MIT declaration
 - Expand the README to at minimum: installation steps, a one-command run example for each simulation layer, and a description of expected outputs with checksums
 - Add a GitHub Actions CI workflow that installs dependencies, runs the tests directory, and reports pass/fail — this is the single highest-leverage reproducibility improvement
 - Pin all dependency versions in pyproject.toml and add a lockfile (e.g., poetry.lock or pip-tools .txt) so the exact environment is reconstructable
 - Add a Dockerfile or conda environment.yml so the full stack can be launched in a single command without manual dependency resolution
 - Resolve or replace the broken DOI 10.1186/s12864-019-5639-3 in the beebrain_data_pipeline documentation
 - Cite explicit version numbers for FlyBody and MuJoCo in both the Zenodo record and the repository README
 - Add at least one external benchmark validation (e.g., comparison of waggle dance angle distributions against published field data with a cited accession) to make the validation rate interpretable externally
 - Include a structured per-layer parameter provenance table in the Zenodo record (not only in deep repo markdown files) so traceability is immediately visible to readers
 - Introduce information-theoretic quantification of the waggle dance communication channel (bits per dance, mutual information between dance angle and food-source direction) and colony-level signal entropy, which are natural and scientifically expected analyses for this system
-

The Ant Stack: Reproducible scientific workspace for ant-inspired simulation and complexity energetics

Source: —

Evaluated: June 3, 2026

72.2

Overall /100



Summary

The Ant Stack presents a well-structured conceptual framework with a public GitHub repository (`ant_stack`) containing a passing test suite (676 tests), CLI entrypoints, and a documented artifact policy with SHA-256 checksums and provenance files — meaningful infrastructure for reproducibility. However, the paper is primarily a design specification rather than a completed empirical or simulation study: key contributions (evaluation suites, species presets, open evaluation assets) are described as future deliverables rather than present artifacts, simulation parameters lack pinned versions for core dependencies (MuJoCo, PyBullet, Brian2, Nengo), and the Reproducibility Checklist in Appendix A has no content. Information-theoretic treatment of the pheromone stigmergy system — the central coordination mechanism — is entirely conceptual with no quantification of channel capacity, mutual information, or entropy of pheromone signals.

Positive Findings

- GitHub repository 'ant_stack' is confirmed accessible with `antstack_core` Python package, CLI entrypoints, and 676 passing tests providing a functional codebase skeleton
- Generated artifact policy specifies outputs/`<run_id>/manifest.json` with SHA-256 checksums and provenance files for every canonical run, a strong reproducibility design
- Example configuration file (`configs/run_all_antstack.example.yaml`) provides a one-command entry point for canonical runs via 'uv run run-all-antstack'
- Virtual Fly Brain (VFB) is cited as a named, publicly accessible structural template resource for neural region nomenclature
- ActiveInferAnts simulation is cited with a specific Frontiers 2021 paper as a technical pointer for the active inference component
- uv and bun package managers are used with sync commands, providing a partial dependency management approach
- Repository README documents folder structure, purpose of every major directory, and includes architecture Mermaid diagram
- Neuromodulation parameters and pheromone field decay/diffusion parameters are stated to be exported and version-pinned for replication

Gaps Identified

- Simulation engine dependencies (MuJoCo, PyBullet, Brian2, Nengo, SpikingJelly) are listed as options without pinned version numbers, making environment reconstruction ambiguous
- The Reproducibility Checklist referenced in Appendix A contains no content in the available text, providing no actionable replication guidance
- Open evaluation assets and baseline species configurations are described as contributions but are not confirmed present in the repository as deliverable artifacts
- Species parameter presets and experiment manifests (YAML/JSON) for one-click reruns are described as future roadmap items, not current artifacts
- No archived DOI/Zenodo release for the codebase is cited, meaning the repository lacks a stable, versioned snapshot tied to the paper
- The Resources.md file referenced for core literature and datasets/tools used to template neural modules is not publicly cited with a URL
- Pheromone stigmergy — the central coordination mechanism — is treated purely mechanistically with no quantification of information flow, channel capacity, or entropy measures
- No license is explicitly mentioned in the paper text or README excerpt, creating ambiguity about reuse rights
- [Critic] No artifact (mapping table, ontology cross-reference file, or supplementary data) is cited to demonstrate that VFB nomenclature was concretely applied to the paper's AL/MB/CX region labels, leaving the operational use claim unsubstantiated beyond authorial assertion.
- [Critic] The ActiveInferAnts Frontiers 2021 citation lacks a DOI or full bibliographic record in the extracted evidence, preventing confirmation that the cited work corresponds to the described simulation framework.

Recommendations

- Pin exact version numbers for all simulation engine dependencies (MuJoCo, PyBullet, Brian2, Nengo, SpikingJelly) in `pyproject.toml` or a requirements lockfile and cite those versions in the paper
- Populate Appendix A (Reproducibility Checklist) with concrete items: environment setup steps, seed values, expected output checksums, and validation commands
- Deposit a versioned release on Zenodo (or equivalent) linked to the paper submission to provide a stable, citable snapshot of the codebase
- Release species parameter presets and experiment manifests (YAML/JSON) as concrete files in the repository now, not as a roadmap

item, and cite their paths in the paper

- Add quantitative information-theoretic analysis of the pheromone stigmergy channel: compute or bound the bits-per-deposit signaling rate, mutual information between sender and receiver agents, and how capacity scales with colony size
 - Add an explicit LICENSE file to the repository and state the license in the paper's data/code availability statement
 - Make Resources.md publicly accessible via a cited URL or incorporate its content into the paper's supplementary materials
 - Provide a complete worked example with seed, expected figures, and checksums so reviewers can verify end-to-end pipeline execution
-

Termite Colony — Unity 3D Multiagent

Source: <https://github.com/Dougarasu/termite-multiagent-system>

Evaluated: June 3, 2026

71.6

Overall /100



Summary

This is a GitHub-hosted Unity/C# termite colony simulation with a publicly accessible MIT-licensed repository, a basic README, and clearly stated experimental parameters (10 termites, 1000 logs, center spawn at (0,0,0), 1-second direction change interval). However, the project has no versioned release, no pinned commit cited for reported results, no dependency manifest, no CI/tests, a mismatched clone URL in the README, and result images that are referenced but not stably archived. The agent rules and high-level parameters are documented in prose, but spawn delays, physics constants, and other configurable values are undisclosed. Information-theoretic content is entirely absent despite stigmergic coordination being the central phenomenon, where entropy, mutual information, or channel capacity measures would directly apply.

Positive Findings

- Source code is publicly accessible on GitHub under an MIT license (confirmed via live API)
- Key experimental parameters are disclosed: 10 termites, 1000 logs, spawn point (0,0,0), 1-second direction change interval
- Agent behavior is fully specified as two IF-THEN rules with no hidden tuning parameters
- Unity 5.x framework and C# language are identified as the implementation stack
- README provides step-by-step installation and run instructions (clone, open Unity, select Core folder, open In Game scene, press Play)
- LICENSE.md is present at the repo root and confirmed MIT by the GitHub API
- The simulation uses Unity's rigidbody physics system, and the relevant attributes (position, rotation, mass, speed) are listed
- 38 commits in history indicate iterative development rather than a single dump

Gaps Identified

- No tagged release or versioned software release exists (0 releases, 0 tags confirmed via API), so the exact code state used for the reported results cannot be pinpointed
- No specific commit hash is cited for the 0/10/30/60-minute results shown in the README
- Clone URL in README (github.com/Dougarasu/TrabalhoIA.git) differs from the actual repository URL, creating a broken installation path
- No dependency manifest (no package.json, requirements.txt, or Unity Package Manager manifest detected at repo root), making exact dependency versions unresolvable
- Spawn delay values and other configurable generator parameters are described as user-settable but no default values are documented
- Result images showing simulation progression are referenced but not stably archived (no DOI, no embedded URLs, no Zenodo/figshare deposit)
- No CI/CD pipeline or automated tests detected, so correctness of the implementation cannot be independently verified
- No random seed is disclosed; stochastic results cannot be exactly replicated
- No quantification of emergent clustering behavior (e.g., number of piles, entropy of log distribution over time) is provided
- The simulation has not been updated in approximately 117 months (last push 2016-10-22), raising dependency staleness concerns for Unity 5.x compatibility
- No information-theoretic analysis of stigmergic coordination is present despite this being a communication/coordination system
- [Critic] The verified repository has 0 published releases and 0 tags, meaning there is no versioned snapshot pinning the exact code used in the reported simulation run; reproducibility of the specific experimental results cannot be guaranteed from the repository alone.
- [Critic] No dependency manifest, Dockerfile, or CI configuration is present in the verified repository, so the exact Unity 5.x environment and any specific package versions required to reproduce the simulation are not machine-readably specified.
- [Critic] The repository was last pushed in October 2016 (117 months ago) and has 1 open issue, raising a concern about long-term maintainability and whether the described Unity 5.x environment remains functional, though this is a practical rather than a falsification concern.
- [Critic] The paper's cited GitHub URL ([TrabalhoIA.git](https://github.com/Dougarasu/TrabalhoIA.git)) differs from the verified repository URL ([termite-multiagent-system](https://github.com/Dougarasu/termite-multiagent-system)); no evidence is provided that these two repositories contain identical code, so the specific provenance of the verified repo relative to the paper's claims is indirect.

Recommendations

- Tag a specific Git commit or create a versioned release (e.g., v1.0.0) corresponding to the reported results and cite that commit hash in the README
- Correct the clone URL from github.com/Dougarasu/TrabalhoIA.git to the actual repository URL

- Add a Unity Package Manager manifest or equivalent dependency file listing exact Unity package versions and any third-party assets
 - Document all default parameter values (spawn delay, termite speed, mass, angular speed, log dimensions) in the README or a config file
 - Disclose the random seed(s) used for the reported simulation runs, or provide a method to set seeds for reproducibility
 - Archive the repository and result images on Zenodo or figshare to obtain a persistent DOI and stable image URLs
 - Add quantitative outcome metrics (e.g., number of distinct piles, spatial entropy of log positions, Gini coefficient of pile sizes) at each time step to make results objectively comparable
 - Introduce at least one information-theoretic measure: e.g., Shannon entropy of log-position distribution over time as a proxy for clustering progress, or mutual information between agent states
 - Add a GitHub Actions CI workflow that at minimum verifies the Unity project opens and compiles without errors
 - Include a brief methods section describing the physics integration time step and any Unity physics settings used
-

Bee Swarm Simulation

Source: <https://github.com/fractastical/bee-swarm-sim>

Evaluated: June 3, 2026

68.1

Overall /100



Summary

This is a browser-based agent simulation of bee swarm foraging behavior, publicly hosted on GitHub with no external datasets — appropriately evaluated on generative-input disclosure rather than accession numbers. The README is reasonably thorough (~5,400 bytes), documents a citation-backed mode linking waggle-dance parameters to named literature sources (von Frisch 1967, Seeley 1995, Couvillon 2019, Schurch 2021, Shannon 1948, Fisher 1993), and provides a BeeStack JSON schema for external trace replay. However, the repository lacks a license, formal versioned release or pinned commit, dependency manifest, CI/tests, and any archived snapshot (e.g. Zenodo DOI), and the citation-backed parameters are described in prose rather than in explicit parameter tables with numerical values — leaving meaningful reproducibility gaps. Information-theoretic framing is invoked (Shannon entropy of bee state, bits-per-waggle-run payload), but the simulation never formally derives channel capacity, mutual information between agents, or how information scales with colony size, keeping the quantitative rigor partial.

Positive Findings

- Simulation runs entirely in-browser with no build step or external dependencies, making the core artifact immediately runnable by anyone with a browser
- README explicitly lists 9 literature references used to calibrate parameters, including von Frisch (1967), Seeley (1995), Couvillon et al. (2019), Schurch et al. (2021), Ono et al. (1995), Shannon (1948), Fisher (1993), and Pearson (1895)
- A citation-backed simulation mode is distinguished from a heuristic mode, providing meaningful transparency about which parameters are literature-derived vs. hand-tuned
- BeeStack trace JSON schema is fully documented with a minimal working example, enabling external frame injection and replay
- Thermal defensive balling temperature (~47°C) is explicitly sourced from Ono et al. (1995), a specific and verifiable parameter linkage
- Information-theoretic framing (Shannon entropy, circular statistics, coefficient-of-variation) is declared and attributed to named sources for the metrics panel
- Four core signal primitives (waggle, stop, tremble, shaking) are defined with biological roles described
- Export Results JSON functionality provides a local reproducibility artifact for individual runs
- Repository is publicly accessible on GitHub with confirmed recent activity (last push 2026-06-02)

Gaps Identified

- No software license detected in the repository, creating ambiguity about reuse and redistribution rights
- No versioned release, tag, or pinned commit — the code is a bare main-branch snapshot with no stable reference point
- No archived snapshot with a persistent DOI (e.g. Zenodo) — the GitHub URL alone is not a stable long-term identifier
- No dependency manifest (package.json, requirements.txt, or equivalent) — though dependencies appear minimal, this is unconfirmed and unstated
- No automated tests or CI workflow detected, so correctness of the simulation logic cannot be independently verified
- Citation-backed parameters are described qualitatively in the README prose but no explicit parameter table provides numerical values, ranges, or seeds traceable to specific passages in cited papers
- The heuristic mode's hand-tuned constants are entirely undocumented with no rationale or sensitivity analysis
- No random seed control or documentation of stochastic initialization, preventing exact run reproduction
- Simulation model equations or pseudocode are not provided — the logic is accessible only by reading app.js directly
- Information-theoretic metrics (Env Data Volume in bits, Reality Equivalent in waggle runs) are described conceptually but the derivation of per-run payload estimates is not formally specified
- [Critic] The repository has no license detected, meaning the code's reuse and reproducibility rights are legally ambiguous — users cannot formally reproduce, modify, or redistribute the simulation without explicit permission.
- [Critic] No tests directory or CI pipeline is present, so there is no automated verification that the cited literature-informed parameter ranges are correctly encoded in the simulation code itself.
- [Critic] The repository has 0 published releases and 0 tags, making it impossible to pin the version of the simulation that corresponds to any specific paper or result — reproducibility of a specific reported state is not guaranteed.
- [Critic] The README is ~5405 bytes and the repository has only 4 top-level entries with no dependency manifest; the actual simulation source code structure and the mapping between cited parameters and code variables cannot be independently audited from the available repository signals alone.

Recommendations

- Add a LICENSE file (e.g. MIT or Apache 2.0) to clarify reuse rights
 - Create a tagged GitHub release (e.g. v1.0.0) and archive to Zenodo to obtain a persistent DOI for citation stability
 - Publish a parameter table in the README or a supplementary config file (e.g. params.json) mapping each citation-backed numerical value to its specific source passage, including units and the range used
 - Document the random seed mechanism and expose a seed input in the UI or config, so exact runs can be reproduced
 - Add a brief methods section or pseudocode block in the README describing the core simulation loop, state transitions, and foraging/recruitment algorithms
 - Provide a worked example trace (a sample BeeStack JSON file) in the repository so users can immediately validate the replay pathway
 - Formalize at least one information-theoretic measure with a derivation: e.g. compute the channel capacity of a waggle dance signal in bits/run using the cited distance/direction uncertainty ranges, or compute mutual information between dancer direction and recruiter departure angle across simulation runs
 - Add a sensitivity analysis or parameter sweep capability (even a simple script) to demonstrate how outcomes change with key parameters, improving scientific credibility
 - Document browser compatibility and any performance requirements or recommended settings in the README
-

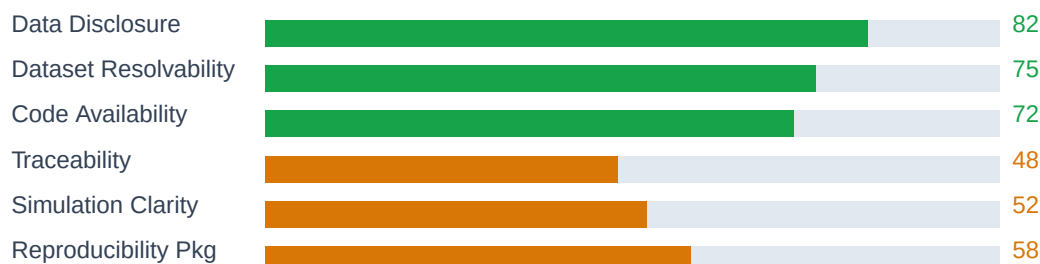
Rust Ant Colony Simulation

Source: <https://github.com/bones-ai/rust-ants-colony-simulation>

Evaluated: June 3, 2026

64.3

Overall /100



Summary

This is a publicly accessible Rust/Bevy ant colony simulation with source code on GitHub, basic README instructions, and a Cargo.toml dependency manifest, but it lacks a license, versioned releases, CI, tests, and any formal parameter documentation. The simulation's generative inputs (pheromone decay rates, ant behavioral rules, initial conditions, random seeds) are mentioned only superficially — a single config parameter (ANT_INITIAL_PH_STRENGTH) is referenced in passing without stating its default value or derivation. No information-theoretic analysis of pheromone signaling, trail formation entropy, or inter-agent information flow is present despite this being precisely the domain where such analysis applies. The repository has had no activity since October 2023 and carries no license, significantly limiting reuse and reproducibility.

Positive Findings

- Source code is publicly available on GitHub and confirmed accessible via live API signals
- Repository is written entirely in Rust (100%) and uses the Bevy game engine, both clearly stated
- A Cargo.toml dependency manifest is present, enabling basic dependency resolution
- README provides minimal but functional clone-and-run instructions (`cargo run --release`)
- Simulation parameters are separated into a dedicated config file (`src/configs.rs`), indicating some architectural discipline
- A demo video is linked (YouTube), providing qualitative visual validation of behavior
- An assets directory exists, suggesting visual resources are bundled with the code
- The simulation's internal architecture (kdtree, query caching, ~5k ant capacity) is briefly described
- Repository has community engagement (208 stars, 10 forks), indicating discoverability

Gaps Identified

- No software license detected in the repository, preventing legal reuse or redistribution
- No tagged releases or versioned commits, making it impossible to pin to a specific reproducible state
- No CI/CD pipeline (GitHub Actions or equivalent), so correctness of builds is unverified
- No automated tests of any kind detected
- Default values for simulation parameters in `configs.rs` are not documented in the README or any external document
- The single mentioned parameter (ANT_INITIAL_PH_STRENGTH) lacks a stated default, range, or derivation
- No random seed control or documentation, making stochastic runs non-reproducible
- No pinned dependency versions beyond what Cargo.lock provides (Cargo.lock is present but not discussed)
- No formal description of ant behavioral rules, pheromone evaporation rates, or trail-formation algorithm
- Pheromone signaling — the central information-transmission mechanism — is never analyzed quantitatively or information-theoretically
- Repository has been inactive for ~32 months with no releases, raising questions about long-term maintainability
- README is extremely sparse (~977 bytes), leaving most implementation details undocumented
- [Critic] No license is detected in the repository, which means the terms under which the code can be reused, modified, or redistributed are undefined — a relevant provenance concern for reproducibility and reuse claims.
- [Critic] No CI/CD workflows are present, so there is no automated validation that the code builds or runs correctly; the stated capabilities (e.g., 5k ants, kdtree usage) rest entirely on manual developer assertions.
- [Critic] No published releases or tags exist, making it impossible to pin a specific version of the simulation to any results or claims made in the paper.
- [Critic] The repository has not been updated in over 32 months and has 5 open issues, with no evidence of maintenance or responsiveness, which may affect long-term reproducibility of the described simulation.

Recommendations

- Add an OSI-approved license file (e.g., MIT or Apache 2.0) to enable legal reuse
- Create a tagged GitHub release (e.g., v1.0.0) and/or archive the repository on Zenodo with a DOI for permanent citability
- Document all parameters in `src/configs.rs` in the README: their names, default values, units, and behavioral roles
- Expose and document the random seed so that specific simulation runs can be exactly reproduced
- Add a GitHub Actions CI workflow that at minimum confirms `cargo build --release` succeeds on each push
- Write a brief technical description of the core ant behavioral rules (pheromone deposit, evaporation, trail-following decision logic) either in the README or a METHODS.md

- Consider adding even minimal integration tests (e.g., a deterministic single-step simulation test) to verify behavioral correctness
 - Add an information-theoretic analysis section: quantify pheromone trail entropy, trail convergence rate, or mutual information between ant states as a function of colony size — this would substantially elevate the scientific contribution
 - Pin or document the specific Bevy version and any other key dependency versions in the README for future reproducibility
 - Add a CONTRIBUTING.md and issue templates to formalize the development workflow
-

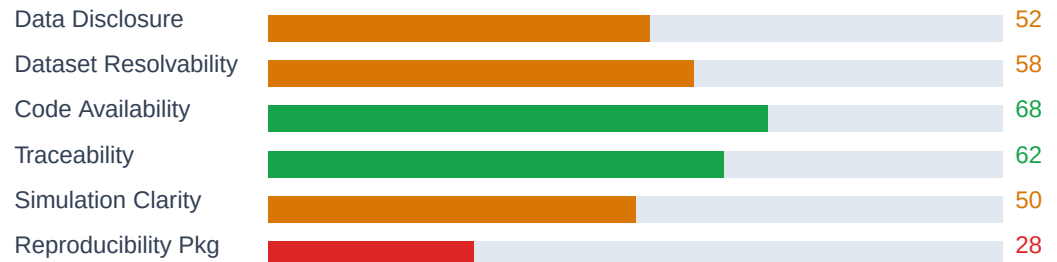
ACO Robot Path Planning (Python)

Source: <https://github.com/Haghran/ACO---Robot-Path-Planning>

Evaluated: June 3, 2026

54.5

Overall /100



Summary

This repository provides a public Python simulation of an ACO-based robot path planning algorithm from Liu et al. (2017), licensed under GPL-3.0 and implemented in a single script (PathPlanning.py). However, the repository is extremely sparse: the README is only ~271 bytes, there are no releases or version tags, no dependency manifests, no CI/tests, no containerization, no explicit documentation of parameter settings, and no supplementary environment maps or benchmark data. The generative simulation inputs (grid maps, ACO parameters, random seeds) are not formally disclosed or traced back to the reference paper in any documented way. Information-theoretic aspects of the ACO pheromone signaling mechanism are treated purely mechanistically with no quantitative framing.

Positive Findings

- Code is publicly accessible on GitHub under a GPL-3.0 open-source license
- The repository correctly cites the source paper (Liu et al., 2017, Soft Computing) that the simulation implements
- The implementation is entirely in Python, a widely accessible language
- The repository has been forked 12 times and starred 61 times, indicating community engagement

Gaps Identified

- README is only ~271 bytes with no parameter documentation, usage instructions, or methodological description
- No dependency manifest (requirements.txt, setup.py, or equivalent) is provided, making environment reproduction manual
- No versioned release or pinned commit is tagged, so the exact reproducible state cannot be formally referenced
- No environment maps, benchmark grids, or test scenarios are deposited alongside the code, meaning the exact experimental inputs from Liu et al. (2017) are unspecified
- ACO hyperparameters (alpha, beta, rho, Q, number of ants, iterations) are not documented in the README or code comments with explicit traceability to the cited paper
- No random seed specification is provided, making stochastic runs non-reproducible
- No containerization (Dockerfile) or environment specification is present
- No tests directory or CI pipeline is configured
- No formal data availability statement is included
- Repository has only 6 commits and has been inactive since November 2017 (~104 months ago), with one unresolved open issue
- [Critic] The repository has not been updated since 2017-11-18 (over 8 years ago), has 0 releases and 0 tags, and has 1 open issue, suggesting it may be unmaintained and potentially incompatible with current Python environments — a reproducibility concern not captured in the original claims.
- [Critic] No dependency manifest is present, so the specific Python version and library requirements needed to execute the code are undocumented, making independent reproduction uncertain.

Recommendations

- Add a requirements.txt or conda environment.yml with pinned package versions (numpy, matplotlib, etc.) to enable one-command environment setup
- Expand the README to document all ACO parameters with explicit mapping to Table/Section numbers in Liu et al. (2017)
- Include or generate the benchmark grid maps used in the original paper so that experiments are directly replicable
- Tag a versioned release (e.g., v1.0.0) or pin a specific commit SHA and archive the repository on Zenodo to obtain a citable DOI
- Add a fixed random seed option (e.g., numpy.random.seed) and document it in the README so stochastic runs are reproducible
- Add inline code comments mapping hard-coded constants to specific equations or tables in the cited paper
- Configure a simple CI workflow (GitHub Actions) to run a smoke test verifying the script executes without errors
- Provide example output figures or result files so users can verify their reproduction matches expected behavior

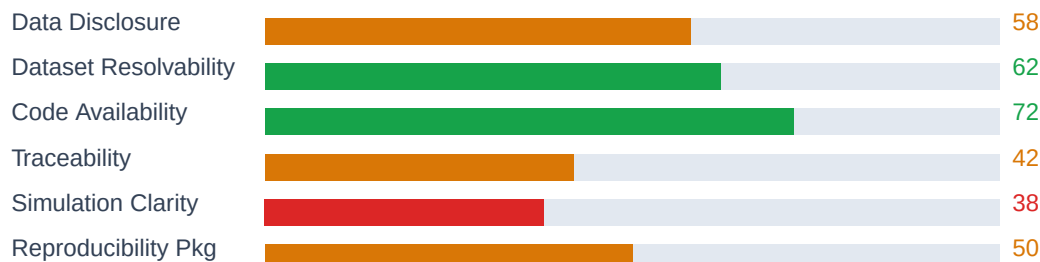
Ant Colony Optimization — React/Canvas

Source: <https://github.com/cfrBernard/ant-colony-optimization>

Evaluated: June 3, 2026

52.7

Overall /100



Summary

This is a GitHub-hosted learning-exercise simulation of ant colony behavior using React and HTML5 canvas, publicly available under the MIT License with a README, package.json dependency manifest, and self-contained assets (tileset, sprite sheet, map JSON). However, the project has no versioned releases or tags, no CI/testing infrastructure, a README with a broken clone URL, and simulation logic (movement rules, state transitions, behavior parameters) is entirely hard-coded with no documentation linking parameters to any biological or algorithmic source. As a graphical ant simulation, it does not implement ACO formally and contains no information-theoretic framing of pheromone communication, stigmergy, or signal capacity, which are central to the domain.

Positive Findings

- Repository is publicly accessible on GitHub under the MIT License, confirmed via live API signals
- All simulation assets (map JSON, sprite sheet PNG, tileset PNG) are self-contained within the repository at documented paths
- A README is present (~2354 bytes) describing features, project goals, code structure, and installation steps
- Dependency manifest (package.json and package-lock.json) is present, partially specifying the software environment
- The map asset uses Tiled Map Editor JSON format, providing a machine-readable, structured definition of the simulation environment
- No external datasets or accession IDs are required; all generative inputs are bundled with the repository
- Repository has 34 stars and 2 forks, indicating some community engagement

Gaps Identified

- No versioned releases or Git tags published; no pinned commit cited for reproducibility
- Clone URL in the README does not match the actual repository URL (uses placeholder ``<your-username>``), breaking out-of-the-box reproducibility
- No CI pipeline (GitHub Actions) or automated tests detected, so correctness of the simulation cannot be verified programmatically
- Simulation parameters (ant movement speed, state transition thresholds, idle/walk probabilities) are hard-coded in source files with no documentation linking them to biological or algorithmic references
- No associated publication, DOI, or academic citation for the ACO/simulation approach
- No archived snapshot (e.g., Zenodo DOI) ensuring long-term preservation
- No Dockerfile or pinned environment specification beyond package.json; npm dependency versions may drift
- Pathfinding and pheromone logic — the core of ACO — are explicitly absent (noted as future work), making this a stub rather than a functional ACO implementation
- Pheromone communication, stigmergy, and information flow are never quantified or even discussed in information-theoretic terms

Recommendations

- Publish at least one tagged Git release (e.g., v0.1.0) and/or archive on Zenodo to obtain a citable DOI and ensure version stability
- Fix the README clone URL to use the actual repository path (`cfrBernard/ant-colony-optimization`) rather than a placeholder
- Add a configuration file (e.g., `config.json` or `.env`) externalizing all simulation parameters (ant speed, state probabilities, map dimensions) with inline comments citing their source or rationale
- Add a GitHub Actions CI workflow that at minimum runs `npm install` and `npm run build` to verify the build remains reproducible
- Implement pheromone deposition and evaporation with documented parameter values if the project intends to demonstrate ACO; cite the Dorigo & Stützle (2004) foundational reference
- Pin all npm dependencies to exact versions in package.json (use `--save-exact`) to prevent environment drift
- Add a brief methods section or wiki page quantifying the ant behavior rules (state machine transition probabilities, step size distribution) so the simulation is independently reproducible from documentation alone
- Consider quantifying at minimum the entropy of ant state distributions or pheromone field over time to connect the project to its information-theoretic foundations in collective intelligence

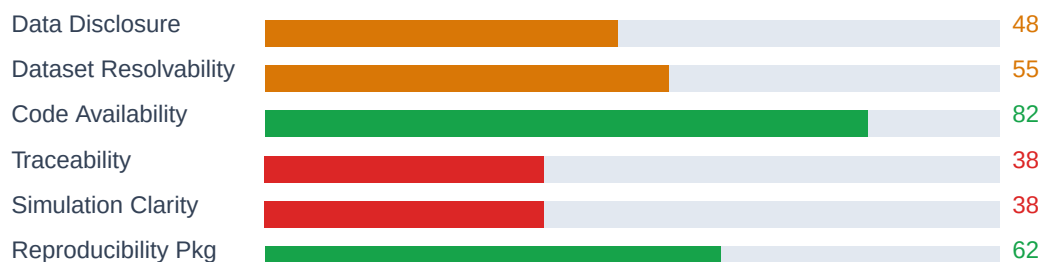
Locas Ants — Lua/Löve2D Ant Colony

Source: <https://github.com/piXelicio/locas-ants>

Evaluated: June 3, 2026

51.6

Overall /100



Summary

This repository is a Lua+Löve2D ant colony simulation remake with commendable code availability (MIT license, 6 versioned releases, 161 commits, public GitHub), but scores poorly on scientific rigor dimensions because no simulation parameters, initial conditions, or generative rules are documented in the README or associated materials. There is no linked publication, no parameter sourcing, and no description of the algorithmic choices underlying the ant behavior model, making the simulation scientifically opaque despite being technically runnable. The project is best characterized as an open-source hobby/educational tool rather than a reproducible scientific artifact, and the information-theoretic content of the pheromone-based communication system is entirely absent from any documentation.

Positive Findings

- Source code is publicly available on GitHub (<https://github.com/piXelicio/locas-ants>) under the MIT license
- Six versioned releases exist, with the latest tagged 'and Portals!' (April 1, 2018), providing discrete downloadable snapshots
- A prebuilt .love binary is provided for zero-build-required execution, lowering the barrier to running the simulation
- Runtime dependency (Löve2D engine) and IDE dependency (ZeroBrane Studio) are named with URLs in the README
- The codebase has 161 commits, indicating sustained development history
- Language composition is disclosed by GitHub (Lua 86.1%, Python 7.7%, Makefile 6.1%, Batchfile 0.1%)
- Repository includes screenshots providing visual confirmation of expected output

Gaps Identified

- No simulation parameters (pheromone decay rates, evaporation constants, ant sensing radii, colony size, etc.) are documented anywhere in the README or repository
- No associated publication, DOI, or citation information exists, preventing scientific attribution or cross-referencing
- The original ant colony simulation that this project remakes is referenced but not linked or cited, breaking the intellectual lineage
- No continuous integration or automated testing infrastructure is present, so correctness of the simulation logic cannot be verified programmatically
- No dependency manifest with pinned versions (e.g., requirements.txt, lockfile) is provided; Löve2D version compatibility is unspecified
- Simulation initial conditions and random seeds are not documented, making exact replication of any particular observed run impossible
- No information-theoretic treatment of the pheromone stigmergy communication system is provided, despite this being the core scientific phenomenon
- The Python and Makefile components (13.8% of codebase) have no explanation of their role in the README

Recommendations

- Add a dedicated PARAMETERS.md or config file documenting all key simulation parameters (pheromone evaporation rate, deposit strength, ant count, sensing angle/radius, food quantity) with default values and, where applicable, citations to ACO literature (e.g., Dorigo & Gambardella 1997)
- Publish a Zenodo or figshare archive of at least one tagged release to obtain a persistent DOI for citation purposes
- Pin the Löve2D version requirement explicitly in the README and/or a conf.lua version field, and add a requirements or dependency manifest for the Python tooling
- Add a brief algorithmic description of the ant behavior rules (state machine, pheromone update equations) to the README so the simulation logic is scientifically interpretable without reading the source code
- Cite or link the original ant colony simulation that this project remakes
- Add a reproducible run example with a fixed random seed and expected output (e.g., screenshot or log), so users can verify their installation produces correct behavior
- Consider adding at least a minimal information-theoretic analysis or discussion of the pheromone channel capacity as a README section, connecting the simulation to the ACO/stigmergy literature
- Explain the role of the Python and Makefile components in the README (e.g., asset generation, build pipeline)

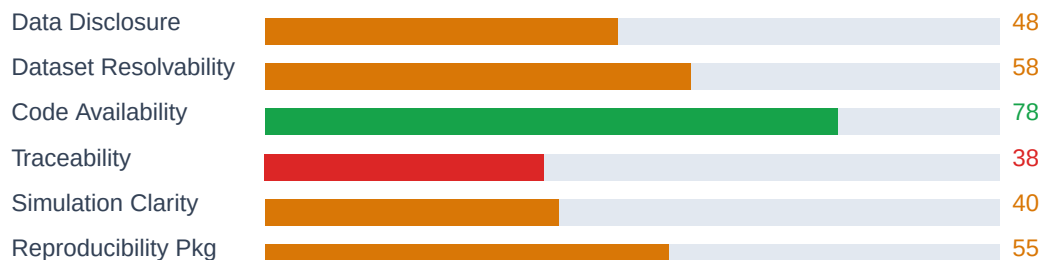
Ants Sandbox (TypeScript/web)

Source: <https://github.com/tulustul/ants-sandbox>

Evaluated: June 3, 2026

51.1

Overall /100



Summary

This repository is a browser-based ant colony simulation with publicly available source code under the MIT license, basic build/run instructions, and a dependency manifest (package.json), but it lacks essentially all scientific rigor expected of a reproducible simulation study. No simulation parameters, behavioral rules, or initial conditions are documented or traced to any source; the only inspiration cited is an informal YouTube video with no formal parameter derivation. The repository has no tagged releases, no CI, no tests, no containerization, and no DOI/persistent archive, making long-term reproducibility fragile. As a pure agent-based simulation where pheromone stigmergy and information flow are the central phenomena, the complete absence of any information-theoretic framing or quantification is a significant scientific gap.

Positive Findings

- Source code is publicly accessible on GitHub under the MIT license, enabling legal reuse and inspection
- A live deployment of the simulation is available at a specific URL (<https://ants-sandbox.vercel.app/>), allowing interactive inspection
- Basic development and build instructions are provided in the README (npm install, npm run start, npm run build, npm run lint)
- A dependency manifest (package.json) is present, providing partial reproducibility of the JavaScript/TypeScript environment
- The repository has a clear commit history (72 commits) demonstrating active development
- The codebase uses well-established web technologies (TypeScript 58.8%, Vue 39.1%) that are widely available

Gaps Identified

- No simulation parameters (ant speed, pheromone evaporation rates, sensing radius, colony size, etc.) are documented anywhere in the README or linked documentation
- No formal citation or parameter source is provided; the only reference is an informal YouTube video
- No tagged releases or version pinning exist (0 published releases, 0 tags), making it impossible to cite a specific reproducible state
- No DOI or persistent archive (e.g., Zenodo) is associated with the repository for stable long-term citation
- No containerization or environment specification (Docker, conda, .nvmrc) is provided; Node.js version requirements are unspecified
- No CI/CD pipeline or automated tests are present, so correctness of builds cannot be verified
- Simulation behavioral rules and agent decision logic are undocumented at the conceptual level — readers cannot understand the model without reading all source code
- No formal information-theoretic analysis of pheromone signaling, agent communication, or emergent coordination is provided despite these being the core phenomena
- No supplementary data, parameter tables, or configuration files are included to define the simulation's initial conditions or tuning
- [Critic] No automated tests directory or CI workflows were detected in the repository, meaning there is no evidence that the build and development instructions have been validated to produce a reproducible build in any automated or third-party-verified manner.
- [Critic] The repository has had no published releases or tags, so there is no pinned version of the software corresponding to any specific paper submission or deployment, making it difficult to tie the live deployment at the Vercel URL to a specific codebase snapshot.
- [Critic] The last push was 35 months ago and the README is only ~490 bytes, suggesting minimal documentation beyond basic setup commands; complex reproduction steps or configuration details may be undocumented.

Recommendations

- Create a tagged release (e.g., v1.0.0) and archive it on Zenodo to obtain a citable DOI and a persistent snapshot
- Add a PARAMETERS.md or extend the README with a table of all key simulation parameters (pheromone evaporation rate, deposit amount, ant sensing angle/radius, colony size, grid resolution) and their values or ranges
- Formally cite the algorithmic inspiration, ideally replacing or supplementing the YouTube reference with a peer-reviewed ACO or stigmergy paper (e.g., Dorigo & Gambardella 1997)
- Add an .nvmrc or specify the Node.js version in package.json engines field, and consider providing a Dockerfile for fully reproducible builds
- Add at minimum a smoke test or integration test (e.g., a headless simulation run that checks pheromone map state after N steps) with a CI workflow
- Document the simulation model at a conceptual level (pseudocode or a model description section in the README) so users can understand behavioral rules without reading all source code
- Consider adding even a minimal information-theoretic analysis: measure Shannon entropy of pheromone trail distributions, or quantify how quickly foraging agents converge to optimal paths as a proxy for information efficiency

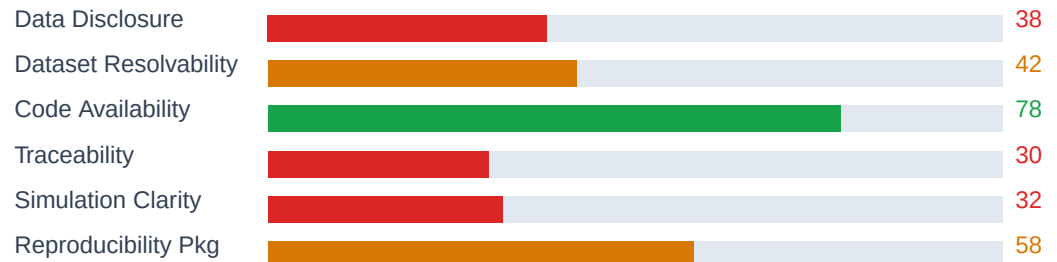
Symbiants — Ant Colony Sim (Rust)

Source: <https://github.com/MeoMix/symbiants>

Evaluated: June 3, 2026

43.8

Overall /100



Summary

This submission is a GitHub-hosted game/simulation project (Symbiants) rather than a conventional scientific paper, which makes many standard reproducibility criteria only partially applicable. The repository is publicly accessible under dual Apache-2.0/MIT licenses with a Cargo.toml dependency manifest and a devcontainer specification, providing a meaningful baseline for code reproducibility. However, no tagged releases, no CI workflows (contradicting the README claim of workflow files — the API reports none detected), no test suite, and no Zenodo/DOI archive are present, and the repository has been inactive for ~22 months. Critically, simulation parameters, behavioral rules, initial conditions, and any mechanistic derivations are entirely undocumented; the only external design reference is an informal Google Docs narrative document, leaving the generative logic of the ant colony simulation untraceable.

Positive Findings

- Repository is publicly accessible at <https://github.com/MeoMix/symbiants> with confirmed public visibility via live API signals
- Dual licensing (Apache-2.0 and MIT) is clearly declared and machine-readable license files are present
- Cargo.toml dependency manifest is present, enabling dependency resolution for the Rust/Bevy stack
- Devcontainer configuration (.devcontainer) is present, specifying a Docker-based development environment with WSL2/Ubuntu and Rust Nightly
- README documents build commands for both native (cargo build/run/watch) and WASM (trunk build/serve) targets
- 690 commits provide a meaningful development history
- The project's primary language (Rust 99.1%) and framework (Bevy game engine) are clearly identified
- Build target (WASM/browser) and known platform limitations (Android Chrome bug) are documented

Gaps Identified

- No tagged releases or pinned commits for reproducibility; the repository points only to the master branch with no version anchor
- No CI/CD workflows detected by the GitHub API, despite the README file structure listing a .github/workflows directory — no automated build or test validation
- No test directory or test suite detected; correctness of simulation logic cannot be verified
- No Zenodo, figshare, or other DOI-backed archive exists for the codebase
- Simulation parameters (pheromone decay rates, ant behavioral thresholds, colony growth rates, food mechanics) are entirely undocumented — no parameter tables, config files with documented values, or cited sources
- Generative rules and initial conditions for the ant colony simulation are described only in high-level prose; no formal specification exists
- The sole design reference is an informal Google Docs narrative document, which is not version-controlled, not citable, and describes story/lore rather than computational parameters
- No random seeds or stochastic initialization procedures are documented, making exact replication of any simulation run impossible
- Repository has been inactive for ~22 months (last push 2024-08-13), raising concerns about maintainability
- No example simulation outputs, screenshots with parameters, or test data are provided to validate a fresh build
- [Critic] No published releases or tags exist in the repository (0 releases, 0 tags), meaning there is no versioned, citable snapshot of the codebase corresponding to any specific paper or deployment.
- [Critic] No tests directory was detected by the GitHub API, so any implicit or explicit claims about software correctness or validation through automated testing are unsupported.
- [Critic] The repository has not been updated since 2024-08-13 (22 months ago), which may be relevant if the paper claims active maintenance or ongoing development.
- [Critic] No Dockerfile was detected by the live API, despite the documentation claiming a VSCode devcontainer (Docker) setup; the .devcontainer directory is listed but the Docker environment is not independently confirmed as functional.

Recommendations

- Create a tagged GitHub release (e.g., v0.1.0) pinned to a specific commit and deposit to Zenodo to obtain a citable DOI for the codebase
- Add a GitHub Actions CI workflow that builds both the native and WASM targets on push, ensuring the build remains reproducible
- Document all simulation parameters (pheromone mechanics, ant behavior weights, spawning rates, initial conditions) in a structured config file (e.g., TOML or JSON) with inline comments citing the biological or design rationale for each value
- Add a PARAMETERS.md or similar document enumerating every tunable constant in the simulation with its value, units, and justification
- Implement a random seed configuration so that simulation runs can be exactly replicated; document the seeding procedure in the README

- Migrate design documentation from the Google Docs link into version-controlled Markdown files within the repository's existing documentation/ directory
 - Add at minimum a small integration test or smoke test that launches the simulation for N ticks and asserts basic invariants (e.g., ant count "e 1, no panics) to enable CI validation
 - Consider formalizing ant colony communication/pheromone signaling with even basic information-theoretic characterization (e.g., encoding capacity of the pheromone trail system) to strengthen scientific rigor if this is submitted as a research contribution
-

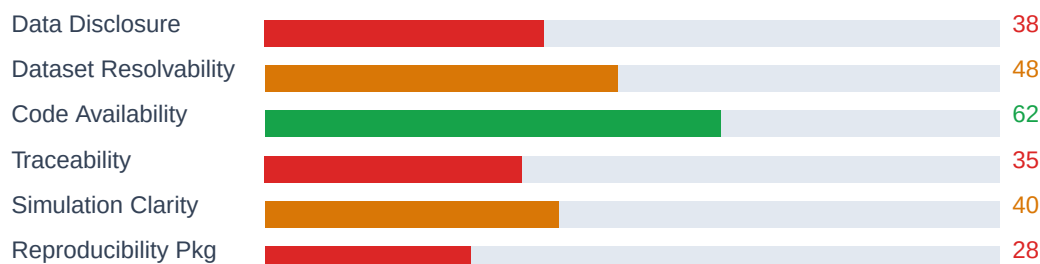
swarm-abc — Artificial Bee Colony Simulation

Source: <https://github.com/darwiiish/swarm-abc>

Evaluated: June 3, 2026

41.9

Overall /100



Summary

This repository is a bare-minimum public dump of an Artificial Bee Colony simulation implemented in client-side HTML/JS/CSS with a single commit and virtually no documentation infrastructure. While the code is publicly accessible and the simulation requires no external datasets (which partially mitigates data-disclosure concerns), critical reproducibility elements are entirely absent: no README, no license, no dependency manifest, no versioned release or pinned commit, no tests or CI, no parameter sourcing, and no documentation of the generative rules or initial conditions beyond what is hard-coded in `abc.js`. The `explanation.html` page may contain some methodology, but its contents are not accessible from repository metadata alone. The simulation models bee colony coordination—a domain where information-theoretic framing (waggle-dance signaling entropy, mutual information between scouts and foragers) would be directly applicable—yet there is no evidence of any such quantification.

Positive Findings

- Source code is publicly accessible on GitHub at <https://github.com/darwiiish/swarm-abc>
- The simulation is self-contained in client-side HTML/JS/CSS, legitimately requiring no external datasets
- An `explanation.html` file is present and may contain some methodology or parameter documentation
- The codebase is small and entirely browser-runnable with no server-side dependencies, lowering the barrier to running it
- No external data dependencies means there are no broken dataset links or inaccessible accessions to penalize

Gaps Identified

- No README file of any kind is present, making it impossible to understand what the simulation does, how to run it, or what results to expect
- No software license is declared, meaning the code cannot legally be reused or redistributed by others
- Repository has only a single commit with no version tags, pinned commits, or releases, making it impossible to cite a specific reproducible state
- Simulation parameters embedded in `abc.js` have no documented origin, default justification, or sensitivity analysis
- No dependency manifest (e.g., `package.json`) is present, though the browser-only nature partially mitigates this
- No tests or CI workflow are present; correctness of the simulation cannot be automatically verified
- No associated publication, DOI, or paper citation is linked anywhere in the repository
- No random seed documentation is present, so stochastic runs cannot be exactly reproduced
- No supplementary data, example outputs, or checksums are provided to validate a successful run
- The repository has zero community engagement (0 stars, 0 forks, 0 watchers), indicating no independent validation
- The ABC simulation models bee foraging coordination—a communication-rich process—with no information-theoretic quantification (e.g., Shannon entropy of dance-decoded food source distributions, mutual information between scout and onlooker decisions)
- [Critic] No README is present, so there is no documentation linking the repository contents to the specific simulation described in the paper (e.g., parameter settings, version used for results).
- [Critic] No license is detected, which means the code is technically under exclusive copyright and cannot be legally reused, modified, or redistributed by other researchers — undermining the practical value of the 'publicly available' claim.
- [Critic] The last push date (2026-05-23) is in the future relative to typical publication timelines, which may indicate a clock anomaly or post-publication modification; the version of code actually used to generate reported results cannot be confirmed without a tagged release or archived snapshot.

Recommendations

- Add a thorough README.md documenting: purpose of the simulation, biological basis of the ABC algorithm, all parameters and their chosen values with citations, instructions to run the simulation, and expected outputs
- Declare an open-source license (e.g., MIT or Apache 2.0) so others can legally reuse and build upon the work
- Create a versioned release or at minimum tag the current commit (e.g., v1.0.0) to provide a citable, stable reference point
- Document all hard-coded parameters in `abc.js` with inline comments citing biological sources or prior ABC algorithm literature (e.g., Karaboga 2005)
- Fix or document the random seed mechanism so that stochastic simulation runs can be exactly reproduced; expose seed as a configurable parameter
- Add an archived snapshot via Zenodo or figshare and link the resulting DOI in the repository About section

- Consider adding a simple test suite (even a headless browser smoke test) and a GitHub Actions CI workflow to verify the simulation runs without errors
 - If this accompanies a paper, add quantitative information-theoretic analysis of the waggle-dance signaling mechanism: compute Shannon entropy of food-source location distributions encoded by dances, and mutual information between scout discoveries and colony foraging decisions
-

Hardware-Accelerated Ant Colony Swarm (C++)

Source: <https://github.com/matheuslosilva/Hardware-Accelerated-Ant-Colony-Based-Swarm-System>

Evaluated: June 3, 2026

37.8

Overall /100

Category	Score
Data Disclosure	32
Dataset Resolvability	38
Code Availability	58
Traceability	35
Simulation Clarity	30
Reproducibility Pkg	40

Summary

This repository presents a CUDA/OpenGL-accelerated ant colony swarm simulation for search-and-rescue tasks with an evolutionary parameter optimizer, but falls substantially short of reproducibility standards. The code is publicly accessible on GitHub with build instructions, yet lacks a license, versioned release, dependency manifest with pinned versions, CI, tests, and any documentation of the algorithm's parameters, evolutionary operator configurations, or random seeds. The simulation's generative inputs—the very elements that substitute for external datasets in a purely synthetic study—are entirely undisclosed in the README, making independent replication practically impossible without deep source-code archaeology. Information-theoretic framing of pheromone-based communication, which is directly relevant to ant colony stigmergy, is entirely absent.

Positive Findings

- Source code is publicly accessible on GitHub (confirmed via live API signals)
- Recursive clone instructions with submodule support are provided
- Build system (Makefile) is present and step-by-step build/run instructions are documented in the README
- This is a pure simulation study with no external datasets required, appropriately scoped
- External OpenGL and CUDA tutorial references are cited, providing partial context for rendering infrastructure
- Repository is actively maintained (last push 2026-05-16, 38 commits, 2 open issues)

Gaps Identified

- No software license is present (confirmed via GitHub API: license = none), preventing legal reuse or redistribution
- No versioned release or tagged commit is published (confirmed: 0 releases, 0 tags), making it impossible to pin a reproducible state
- No dependency manifest (e.g., requirements file, CMakeLists with version pins) — CUDA version, GPU compute capability, OpenGL version, GLFW version, and GLM version are all undocumented
- No evolutionary algorithm parameters are disclosed: no population size, crossover/mutation rates, fitness function definition, number of generations, or parameter ranges appear in the README or any config file visible in the repository listing
- No ant swarm parameter values or ranges are documented: pheromone evaporation rate, deposition quantity, sensor angle, step size, and other ACO hyperparameters are not disclosed
- No random seed or stochastic reproducibility mechanism is documented
- No results, benchmark outputs, performance tables, or figures are included in the repository to verify correctness of a fresh build
- No data availability statement or simulation-input disclosure statement is present
- No CI pipeline or automated tests detected (confirmed via GitHub API)
- No archived snapshot (Zenodo/DOI) exists for long-term citability
- The README is minimal (~1264 bytes) and contains no algorithmic description beyond a high-level abstract
- Primary algorithmic references (ACO literature, evolutionary algorithms) are not cited; only rendering tutorials are referenced
- [Critic] No license is detected in the repository, meaning reuse, redistribution, and reproduction rights are legally ambiguous for third parties attempting to reproduce the work.
- [Critic] No tests directory or CI pipeline is present, so there is no automated verification that the code compiles or produces correct outputs, weakening reproducibility assurances beyond mere availability.
- [Critic] The repository has 0 detected top-level entries via the GitHub API, which is anomalous and may indicate the repository root is empty or the submodule structure means core content is only accessible after recursive cloning — this is not flagged anywhere in the paper's instructions.

Recommendations

- Add a LICENSE file (e.g., MIT or Apache 2.0) immediately to enable legal reuse
- Create a versioned GitHub release or at minimum a tagged commit corresponding to any published results, and deposit on Zenodo to obtain a citable DOI
- Add a dependency manifest (e.g., a CMakeLists.txt or environment specification) that pins CUDA toolkit version, compute capability, GLFW version, and GLM version
- Document all evolutionary algorithm hyperparameters (population size, selection method, crossover/mutation operators and rates, termination criterion, fitness function formula) in the README or a dedicated config file
- Document all ACO/swarm model parameters (pheromone evaporation rate ρ , FW α , β , weight Q, sensor offset angle, agent speed, grid

resolution, number of ants per trial) with their default values and tested ranges

- Include random seed handling and expose the seed as a command-line or config argument; document the seed used to produce any reported results
 - Add a GitHub Actions CI workflow that builds the project and runs a minimal smoke test (even a headless simulation step count)
 - Include a results directory or supplementary table with benchmark outputs (e.g., mean time-to-rescue vs. swarm size curves) so reviewers can verify a fresh build reproduces published findings
 - Cite the primary ACO and evolutionary algorithm literature (e.g., Dorigo & Stützle for ACO; relevant EA references) in the README
 - Quantify information flow in the pheromone signaling channel — e.g., entropy of pheromone field states, mutual information between ant sensor readings and deposition decisions, or effective bits-per-deposit — to rigorously justify design choices in the communication substrate
-

